

[Edit](#)

Build a Real Project by Using Git Workflows

Challenge Overview

Understand the scenario

You are a Developer for Hexelo, an organization that needs to simulate a full Git workflow on a real project.

In this Challenge Lab, you will simulate a full development cycle on a Git project by initializing a repository, creating files and committing changes, branching, merging, and pushing the repository to GitHub. First, you will create all the starter files you need to complete the project. Next, you will commit files and push to GitHub. Next, you will merge a new branch into the main branch, and then you will view history and differences. Finally, you will resolve a merge conflict.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

🔗 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Configure your environment and create a starter project

Hints Enabled

No Yes

- Sign in to the virtual machine as **LabUser** using T **Passw0rd!** as the password.
- Enter your GitHub Username, Email Address, Password, and GitHub Account URL into the following associated textboxes:

GitHub Username

GitHub Email Address

GitHub Password

GitHub URL

- Open **Microsoft Edge**, go to T <https://github.com>, and then sign in as T **<GitHubEmail>** using T **<GitHubPW>** as the password.



Expand this hint for guidance on signing in to a GitHub account.



- On the Windows taskbar, select the **Microsoft Edge** button.
- In the Address bar, enter T <https://github.com>, and then press **Enter**.
- On the GitHub home page, in the upper right-hand corner, select **Sign in**.
- On the *Sign in to GitHub* page, in Username or email address, sign in as T **<GitHubEmail>** using T **<GitHubPW>** as the password.
- If presented with the *Save password* dialog, select **Never**.
- If presented with the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.



Want to learn more? Review the documentation on [signing in to a GitHub account](#).

- Update the **git-bash** Application located in the `C:\Program Files\Git` folder by using the `git update-git-for-windows` command.



Expand this hint for guidance on updating git-bash.



- In the Windows taskbar menu, select **File Explorer**.
- In File Explorer, in the Address bar, enter `C:\Program Files\Git`, and then press **Enter** on the keyboard.
- In the Git folder, double-click the **git-bash** Application to open it.
- Run the following command to download and install Git:

```
git update-git-for-windows
```

- After running the command, enter **Y** at the prompt, and then select **Yes** on the User Account Control pop-up. The current git-bash window will close, and the Git Setup window will launch in the background (it may be behind other windows).
- Once the install is complete (the Git Setup window will close), double-click on the **git-bash** Application to re-open it.



Want to learn more? Review the documentation on [installing git](#).

- Set your Git Identity by using the `git config` command and using `<Name>` as the **username** and `<GitHubEmail>` as the **email address**.



Expand this hint for guidance on setting your Git Identity.



- In the Git Bash Terminal, run the following commands to set your Git identity-press **Enter** on your keyboard after entering a command.

```
git config --global user.name "<Name>"
```

```
git config --global user.email "<GitHubEmail>"
```



Want to learn more? Review the documentation on [setting your Git Identity](#).

 The `--global` Git identity can be changed as many times as you like (each change overwrites the previous one), and you can always override it for individual repositories using `--local`.

- Create a project folder named `todo-app{LAB_INSTANCE_ID}` in the `D:\LabFiles` folder by using the `mkdir` command.

 Expand this hint for guidance on creating a new project folder. 

- In the Git Bash Terminal, run the following command to create a project folder:

```
mkdir /d/LabFiles/todo-app{LAB_INSTANCE_ID}
```

- Navigate to the `todo-app{LAB_INSTANCE_ID}` project folder by using the `cd` command.

 Expand this hint for guidance on changing folders. 

- In the Git Bash Terminal, run the following command to change to the project folder:

```
cd /d/LabFiles/todo-app{LAB_INSTANCE_ID}
```

 Want to learn more? Review the following command documentation:

- [mkdir](#)
- [cd](#)

- Run the following command to initialize the `todo-app{LAB_INSTANCE_ID}` folder as a Git repository:

```
git init
```

 Want to learn more? Review the documentation on the [git init](#) command.

- Create a file named `index.html` in the `D:\LabFiles\todo-app{LAB_INSTANCE_ID}` folder that contains the following HTML code by using `notepad`:

```
index.html
```



```
<!DOCTYPE html>
<html>
<head>
<title>ToDo-App</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1>My To-Do List</h1>
<ul id="todo-list"></ul>
<input type="text" id="new-task" placeholder="New task">
<button onclick="addTask()">Add</button>
<script src="script.js"></script>
</body>
</html>
```



Expand this hint for guidance on creating a file.



- In the Git Bash Terminal, run the following command to open a new file named index.html using Notepad:



```
notepad index.html
```

- On the pop-up window that says *Do you want to create a new file?*, select **Yes**.
- Use the  **Copy** icon to copy the text below to the clipboard, and then in Notepad, enter **Ctrl + V** on the keyboard to paste.

```
index.html
```



```
<!DOCTYPE html>
<html>
<head>
<title>ToDo-App</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1>My To-Do List</h1>
<ul id="todo-list"></ul>
<input type="text" id="new-task" placeholder="New task">
<button onclick="addTask()">Add</button>
<script src="script.js"></script>
</body>
</html>
```

- After pasting the text, enter **Ctrl + S** on the keyboard or select **File > Save**.
- Close **Notepad**.

⚠ Use the **Copy to clipboard** option to enter code for the HTML, CSS, and JS documents, as the **Type Text** option may introduce additional spaces or breaks, which will malform the code.

- Create a document named **style.css** in the **D:\LabFiles\todo-app{LAB_INSTANCE_ID}** folder by using **notepad** that contains the following HTML code:

style.css

```
body { font-family: Arial; margin: 40px; }  
li { margin: 5px 0; }
```

- Create a file named **script.js** in the **D:\LabFiles\todo-app{LAB_INSTANCE_ID}** folder by using **notepad** that contains the following HTML code:

script.js

```
function addTask() {  
  const task = document.getElementById('new-task').value;  
  const list = document.getElementById('todo-list');  
  if (task.trim() !== '') {  
    const item = document.createElement('li');  
    item.innerText = task;  
    list.appendChild(item);  
    document.getElementById('new-task').value = '';  
  }  
}
```

Check your work

Verify

Commit files and push to GitHub

Hints Enabled

No Yes

- Create your first commit by entering the following code into Git Bash:

```
echo "# todo-app" > README.md && git add README.md && git commit -m "Initial c
```

 You can chain commands like `echo > file && git add file && git commit -m "message"` to streamline work, but beginners should do each one step-by-step until comfortable.

You can check for commits by using the `git log` command.

 Want to learn more? Review the documentation on the following commands:

- `echo`
- `git add`
- `git commit`

- Create a new repo on GitHub named `T` `todo-app{LAB_INSTANCE_ID}`.

💡 Expand this hint for guidance on creating a new repo on GitHub.

- Open a new **Microsoft Edge** browser tab, and then go to <https://github.com/new>.

- On the *Create a new repository* page, in Repository name, enter [todo-app{LAB_INSTANCE_ID}](#), and then select **Create repository**.

💡 Want to learn more? Review the documentation on [creating a new repo on GitHub](#).

- In the Git Bash Terminal, add a new remote URL named [origin](#) by running the following command:

```
git remote add origin <GitHubURL>/todo-app{LAB_INSTANCE_ID}.git
```

- Ensure that you have a remote origin URL set by using the [git remote](#) command.

💡 Expand this hint for guidance on verifying the remote origin URLs.

- Run the following command to confirm that you have a remote URL set:

```
git remote -v
```

💡 Want to learn more? Review the documentation on [managing tracked repositories](#).

- Rename the branch to main by running the following command:

```
git branch -M main
```

- Review the status of the working directory by running the following command:

```
git status
```

 You should see the starter files created earlier listed as Untracked files: *index.html*, *style.css*, and *script.js*.

- Stage all changed files by running the following command:

```
git add .
```

- Commit your files by running the following command:

```
git commit -m "Created starter files"
```

- Push the `main` branch to the remote repository on GitHub by using the `git push` command, and then if prompted, sign in to GitHub with your browser.

 Expand this hint for guidance on pushing to GitHub. 

- Run the following command to push to GitHub:

```
git push -u origin main
```

- On the GitHub sign in page, select **Sign in with your browser**.
- If prompted, on the Authorize Git Credential Manager page, select **Authorize git-ecosystem**.

 Want to learn more? Review the documentation on [pushing to GitHub](#).

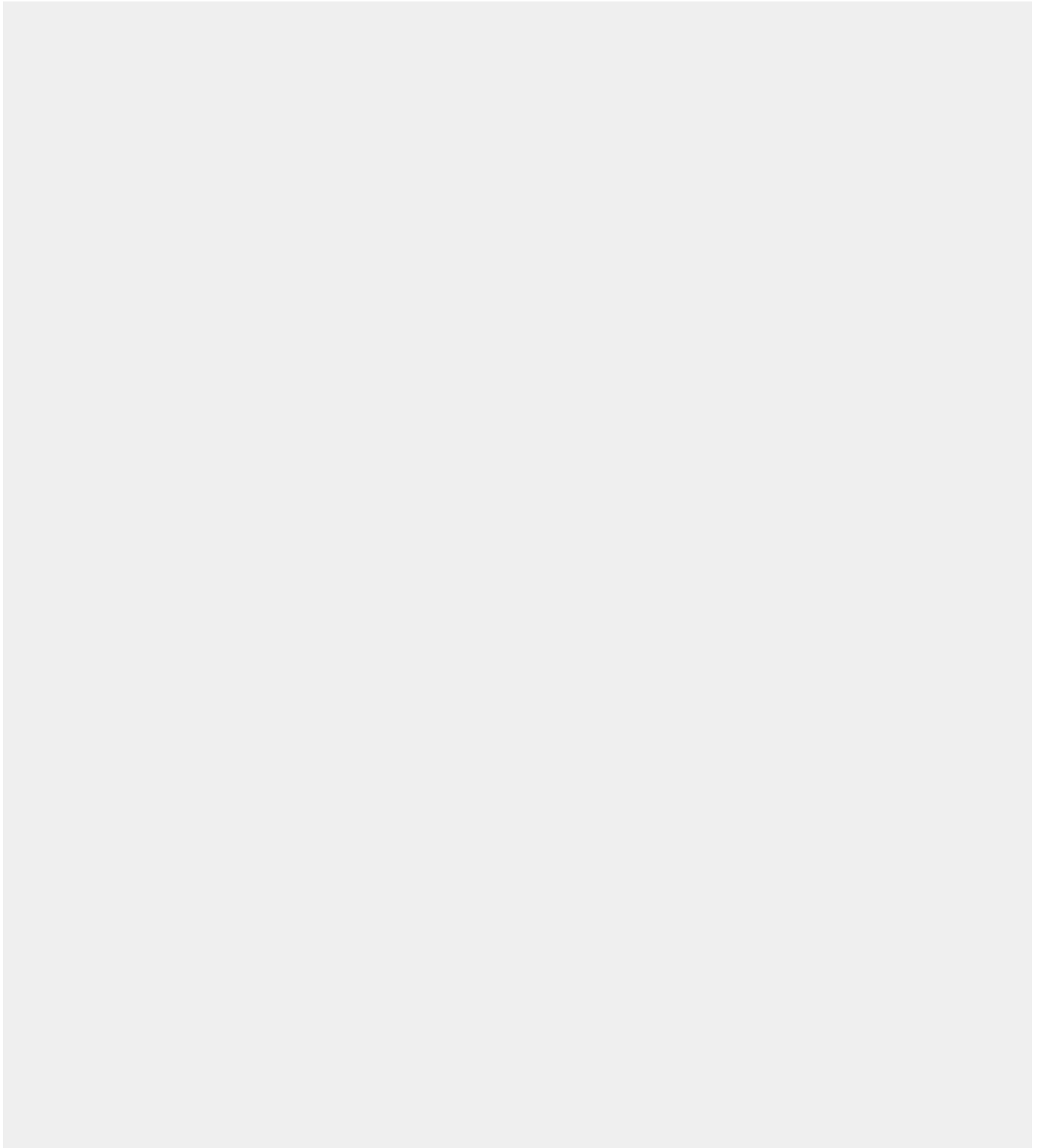
 Once you've connected your local repo to GitHub, you can use *git push* for future updates without *-u origin main*. Git remembers the upstream branch after the first push.

If your push fails, it's often because of a mismatch or missing remote. Use *git remote -v* to confirm.

Check your work



Verify



Create a new branch and merge changes into main

Hints Enabled

No Yes

- Create a new branch named `feature-welcome-message` by using the `git branch` command.

💡 Expand this hint for guidance on creating a new branch. ^

- Run the following command to create a new branch:

```
git branch feature-welcome-message
```

💡 Want to learn more? Review the documentation on the [git branch](#) command.

- Switch to the `feature-welcome-message` branch by using the `git switch` command.

💡 Expand this hint for guidance on switching to a new branch. ^

- Run the following command to switch to a new branch:

```
git switch feature-welcome-message
```

💡 Want to learn more? Review the documentation on [switching to a new branch](#).

📄 Use descriptive names for branches, like *feature-welcome-message*, so collaborators understand its purpose.

- Create a `hello.txt` file and add a line that contains the text `"Welcome to branching in Git!"` by using the `echo` command.

💡 Expand this hint for guidance on creating a file with text. ^

- Run the following command to create a new file:

```
echo "Welcome to branching in Git!" > hello.txt
```

- Confirm the file has the appropriate text by using the `T` `cat hello.txt` command.

💡 Want to learn more? Review the documentation on the [cat](#) command.

- Stage the `T` `hello.txt` change by running the `T` `git add` command.

💡 Want to learn more? Review the documentation on the [git add](#) command.

- Commit the change by using the `T` `git commit -m` command and `T` `"Added welcome message to hello.txt"` as the message.

💡 Expand this hint for guidance on committing a change. ^

- Run the following command to commit the change:

```
git commit -m "Added welcome message to hello.txt"
```

💡 Want to learn more? Review the documentation on [committing a change](#).

📄 Always commit before switching branches. Uncommitted changes stay in your working directory and may cause confusion.

- Switch back to `T` `main` by using the `T` `git switch` command.

💡 Expand this hint for guidance on switching back to main. ^

- Run the following command to switch to main:

```
git switch main
```

- Merge the `feature-welcome-message` branch by using the `git merge` command.



Expand this hint for guidance on merging the feature branch.



- Run the following command to merge the branch:

```
git merge feature-welcome-message
```



Want to learn more? Review the documentation on [merging the feature branch](#).



The results should look like the following:

```
Updating abc1234..def5678
Fast-forward
hello.txt | 1 +
```

- Delete the `feature-welcome-message` branch by using the `git branch` command:



Expand this hint for guidance on deleting a branch.



- Run the following command to delete the branch:

```
git branch -d feature-welcome-message
```



Want to learn more? Review the documentation on [deleting a branch](#).



After a successful merge, you should delete your feature branch locally with `-d`, but use `-D` if it wasn't merged and you want to force deletion.

Check your work

Verify

View history and differences

Hints Enabled

No Yes

- View the commit history by using the `T` `git log` command.

💡 Expand this hint for guidance on viewing the commit history. ^

- Run the following command to view the commit history:

```
git log --oneline
```

💡 Want to learn more? Review the documentation on [displaying the commit history](#).

📄 The `git log --oneline` command is a cleaner way to review commit history; use it frequently to understand your project timeline.

- View the file difference by using the `T` `git diff` command with `T` `HEAD~1` to see what changed recently.

💡 Expand this hint for guidance on viewing the file difference. ^

- Run the following command to view the file difference:

```
git diff HEAD~1 HEAD
```

💡 Want to learn more? Review the documentation on [viewing the file difference](#).

📄 Use `HEAD~1` to compare with the previous commit, but you can also use specific commit hashes for more precise diffs. ^

Use `git log --graph` to visually understand how branches diverged and merged.

Add `--color` to many Git commands for better visual clarity (`git log --oneline --color`).

- Repeat the previous command and save the results to the file `diff.txt` located at `D:\LabFiles` by running the following command:

```
git diff HEAD~1 HEAD > "D:\LabFiles\diff.txt"
```

Check your work

Verify

Resolve a merge conflict

Hints Enabled

No Yes

- Create and switch to a new branch named `conflicting-branch` by running the `git switch` command.

💡 Expand this hint for guidance on creating and switching to a new branch. ^

- Run the following command to create and switch to a new branch:

```
git switch -c conflicting-branch
```

📄 Use a different branch name such as *conflicting-branch* for practice to avoid affecting other work.

- To create a conflict with the main branch, modify the `hello.txt` file with the text `This line will conflict with main` by using the `echo` command. Overwrite the file contents instead of appending data.

💡 Expand this hint for guidance on overwriting file contents. ^

- Run the following command to overwrite the file:

```
echo "This line will conflict with main." > hello.txt
```

- Stage the change to the `hello.txt` file by using the `git add` command.
- Commit the change by using the `git commit` command with `"Conflicting change in conflicting-branch"` as the message.
- Switch back to the `main` branch by using the `git switch` command.
- Overwrite the `hello.txt` file contents, stage the change, and commit the change by running the following commands:


```
echo "This line is from main and will also conflict." > hello.txt
```

```
git add hello.txt
```

```
git commit -m "Conflicting change in main"
```

- Attempt to merge the `conflicting-branch` into `main` by running the following command:

```
git merge conflicting-branch
```

 You will see an error:

Auto-merging hello.txt

CONFLICT (content): Merge conflict in hello.txt

Automatic merge failed; fix conflicts and then commit the result.

- Manually edit the `hello.txt` file to delete the conflict markers and enter `Final version after resolving conflict.` as the new content.

 Expand this hint for guidance on deleting conflict markers. 

- Open the `hello.txt` file by running the following command:

```
notepad hello.txt
```

- In the `hello.txt` file, press **Ctrl + A** to select all, delete the contents, and then enter the text `Final version after resolving conflict..`



- Save the file and close **Notepad**.

 Don't panic when you see conflict markers like `<<<<<<`, `=====`, and `>>>>>>`—they show exactly where Git needs help. 

Only remove conflict markers, not the actual data you want to keep—make sure the final version has only the resolved lines.

- Stage the `hello.txt` changes by using the `git add` command.
- Commit the changes by using the `git commit` command and `"Resolved merge conflict between main and conflicting-branch"` as the message.



Want to learn more? Review the documentation on [branching and merging](#).



You have resolved your first Git merge conflict.

After resolving, always run `git add` and then `git commit` to finish the merge; otherwise, your repository remains in a conflicted state.

- Delete the `conflicting-branch` branch by using the `git branch -d` command.
- Use the `git push` command to push to GitHub, sign in to GitHub, if necessary, and then **Authorize git-ecosystem**.

Check your work

Verify

Summary

Congratulations, you have completed the **Build a Real Project by Using Git Workflows** Challenge Lab.

You have accomplished the following:

- Configured your environment and created a starter project.
- Committed files and pushed to GitHub.
- Created a new branch and merged changes into main.
- Viewed history and differences.
- Resolved a merge conflict.

Submitting your lab

To ensure your lab is recorded as complete, select **Submit** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Track Local Changes and Manage Commits [Guided]
- Create Branches and Merge Changes Safely [Guided]
- Collaborate Remotely and Use Pull Requests [Guided]
- Build a Real Project by Using Git Workflows [Advanced]

- Write Code with GitHub Copilot Suggestions [Guided]
- Control Git Behavior with Config and Ignore Files [Guided]
- Rebase, Stash, and Apply Commits Precisely [Guided]
- Manage Tasks by Using GitHub Issues and Boards [Guided]
- Automate Testing and CI with GitHub Actions [Guided]
- Review and Refactor Code with Copilot [Guided]
- Implement Release Management and Versioning in GitHub [Expert]

